Two heaps in one

Appendix

A heap has a baseline that stays and a volatile part that churns.

- Baseline: the live objects your collections hold between calls.
- Volatile: short-lived garbage that each call allocates and drops.
- The baseline sets the floor; the volatile part sets how often GC runs.
- Both have to fit under `-Xmx` at the busiest moment.

Taking the histograms

Appendix

Two everyday JVMs on a developer laptop, and what they hold.

```
jps -l                          # find the pids
jmap -histo <pid> | head -15    # IntelliJ IDEA
jmap -histo <pid> | head -15    # SonarQube for IDE
```

- Without `:live` the histogram also counts garbage not yet collected.
- Look for `Integer`, `HashMap$Node`, `Object[]` and `$$Lambda` near the top.

Growing a collection leaves slack and garbage

Appendix

A list grown from empty is bigger than a presized one, and it dropped arrays on the way.

- `ArrayList` grows by 1.5x: up to 7% unused capacity in this run.
- Every array it outgrew became garbage; JOL only sees the final list.
- `HashMap` tables double: 46 to 50.5 B per entry, depending on n.
- Presize when the size is known: `new ArrayList<>(n)`, `HashMap.newHashMap(n)`.

Compact headers shrink nodes, not boxes

Appendix

`-XX:+UseCompactObjectHeaders` saves 4 B per header, but objects round up to 8 B.

Retained size	Headers off	Headers on	Saved
`HashMap<Integer, Integer>`, 1 M entries	56.4 MB	48.4 MB	14%
`HashSet<Integer>`, 1 M elements	56.4 MB	48.4 MB	14%
`ArrayList<Integer>`, 1 M, presized	20.0 MB	20.0 MB	0%
`Stream.filter().map().mapToLong()`	304 B	272 B	11%
one `Integer`	16 B	16 B	0%

Garbage you cannot read from the code

Appendix

The same line of code leaves different garbage under different conditions.

- `Integer.valueOf(1)` is cached; `Integer.valueOf(500)` costs 16 B per call.
- Escape analysis removes every EC object, and none of the stream's.
- Compact headers save 16 to 24 B per call; the objects stay.
- `anyMatch(p)` is 40 B cheaper than `filter(p).findFirst()`.

Escape analysis is not a promise

Appendix

With `-XX:-DoEscapeAnalysis` each lambda shape shows what it creates.

`anyMatch`, match at element 500	**EA on**	**EA off**
JDK `list.stream().anyMatch(…)`	176 B	176 B
EC lazy `asLazy().anySatisfy(…)`	0 B	48 B
EC capturing `Iterate.anySatisfy(…)`	0 B	32 B
EC fat-free `Iterate.anySatisfyWith(…)`	0 B	16 B

- The JIT removes allocations only in code it inlines.
- Turning it off is an upper bound, not a typical JVM.

Fat-free lambdas leave less behind

Appendix

```
// capturing: a new lambda on every call, holding the value
Iterate.anySatisfy(list, Integer.valueOf(test)::equals);

// fat-free: one constant lambda, the value is a parameter
Iterate.anySatisfyWith(list, Integer::equals, test);

// JDK stream: the pipeline is built whatever the lambda
list.stream().anyMatch(Integer.valueOf(test)::equals);
```

- Escape analysis off: 32 B capturing, 16 B fat-free, 176 B stream.
- Less garbage per call keeps the headroom free for the next spike.